

# Programming for Robotics: Project Report

Project Title: Autonomous Irrigation Unit (A.I.U)

GitHub Repository Link: <https://github.com/DobTechTD/3003ICT-Project-AIU-Group-14>

Video Link: <https://youtu.be/YA3cA9fydmg>

Group Members:

- Thomas Dobson – s5434264

Specialisation Option Chosen:

☒ Embedded System (Wokwi)

## 1. Project Overview

The Autonomous Irrigation Unit (or A.I.U) is a Arduino Uno embedded controller simulated in Wokwi. The system monitors soil moisture, temperature, light level, and tank water level to manage and maintain irrigation and protect the pump from unsafe indefinite operation.

The system uses sensor fusion, a five-state FSM, adaptive polling, day/night heat-risk adjustment, low-water lockout, and failsafe protection throughout the project. Irrigation only occurs when soil is dry enough, temperature indicates increased irrigation demand, water level is safe, and sensor readings are valid. The intended environment is a small scaled agricultural or greenhouse system where irrigation should respond automatically to changing conditions and allows for user independence and safety.

## 2. Technical Requirements Compliance

| <b><i>Requirement</i></b>    | <b><i>Implementation</i></b>                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Wokwi embedded system</b> | Arduino Uno simulation with environmental component inputs and output devices.                                                            |
| <b>Inputs</b>                | DHT22 temperature sensor, photoresistor for day/night, soil moisture slider potentiometer, water level potentiometer, and the pushbutton. |
| <b>Outputs</b>               | Relay/pump, RGB LED, buzzer, LCD 2004 I2C visual display, and the Serial Monitor for wokwi.                                               |
| <b>Safety</b>                | Low-water lockout, failsafe, sensor validity check, timeout and hysteresis thresholds.                                                    |
| <b>Advanced behaviour</b>    | Sensor fusion, adaptive polling rates, adaptive day/night heat-risk thresholds.                                                           |
| <b>Code quality</b>          | Modular Arduino tabs for ease-of-use configuration, sensors, FSM, outputs, display, and button input                                      |
| <b>Behaviour model</b>       | Five-state FSM: IDLE, MONITORING, IRRIGATING, WARNING, FAILSAFE                                                                           |
|                              |                                                                                                                                           |

### 3. System Architecture

Sensors → Processing → Decision → Act / Outputs

DHT22 temperature sensor, soil moisture potentiometer, water level potentiometer, photoresistor/light sensor, and pushbutton → Arduino Uno processing: it reads sensor values, map analogue inputs, check sensor validity, apply hysteresis, apply day/night context, and adjust adaptive polling → FSM decision logic: IDLE, MONITORING, IRRIGATING, WARNING, and FAILSAFE states using initial conditions sensor fusion, timeout logic, low-water lockout, and failsafe conditions → Relay Module/pump, RGB LED state indicator, buzzer warning output, LCD 2004 I2C display, and Serial Monitor.

### 4. Behaviour Model

| <u>State</u>      | <u>Purpose</u>                | <u>Main outputs</u>                  |
|-------------------|-------------------------------|--------------------------------------|
| <b>IDLE</b>       | Safe startup/standby          | Pump off, buzzer off, blue LED       |
| <b>MONITORING</b> | Normal sensor checking        | Pump off, green LED, LCD updates     |
| <b>IRRIGATING</b> | Active irrigation             | Pump on, cyan LED                    |
| <b>WARNING</b>    | Non-critical unsafe condition | Pump off, yellow LED, warning buzzer |
| <b>FAILSAFE</b>   | Critical safety lockout       | Pump off, red LED, fast buzzer       |

| <u>Transition</u>                       | <u>Trigger</u>                             |
|-----------------------------------------|--------------------------------------------|
| <b>IDLE → MONITORING</b>                | The button pressed                         |
| <b>MONITORING → IRRIGATING</b>          | Dry soil + high temperature + safe water   |
| <b>MONITORING//IRRIGATING → WARNING</b> | Low water, severe heat/dryness, or timeout |
| <b>WARNING → MONITORING</b>             | Warning condition clears                   |
| <b>ANY ACTIVE STATE → FAILSAFE</b>      | Critical water level or invalid sensor     |
| <b>FAILSAFE → IDLE</b>                  | Water recovered and button pressed         |

An FSM is used because the system has clear operating modes with different outputs, safety rules, and transition conditions. As well as the system only needs to be active in one state at one time.

### 5. Perception / Sensor Processing

| <u>Sensor</u>                      | <u>Data read/ Processing</u> | <u>Decision affect</u>                                |
|------------------------------------|------------------------------|-------------------------------------------------------|
| <b>DHT22</b>                       | Temperature in °C dgreess    | Determines high temperature and heat warning flags    |
| <b>Soil moisture potentiometer</b> | Soil moisture percentage     | Determines dry and severely dry conditions            |
| <b>Water level potentiometer</b>   | Tank water percentage        | Determines low-water warning and critical failsafe    |
| <b>Photoresistor</b>               | Light percentage             | Determines day/night context                          |
| <b>The pushbutton</b>              | Press event/ Debounced input | Starts system, acknowledges warnings, resets failsafe |

Sensor fusion was chosen and used because irrigation does not depend on one singular input alone. The system only enters IRRIGATING if and when soil is dry, temperature is high, water is safe, all relevant sensor readings are valid, and the irrigation cooldown has finished or been dismissed manually. The light sensor provides day/night context by changing the heat-risk thresholds. Hysteresis prevents rapid switching near thresholds, such as soil becoming dry below 40% but not clearing until it rises above 50% or water splashing from side to side in the tank from 10% and  $25 \pm 5\%$ .

## 6. Safety and Robustness

| <b><i>Feature</i></b>           | <b><i>Description</i></b>                                                       |
|---------------------------------|---------------------------------------------------------------------------------|
| <b>Low-water lockout</b>        | Pump is disabled when water is low-threshold.                                   |
| <b>Critical-water failsafe</b>  | Critical water level or invalid sensor readings force FAILSAFE.                 |
| <b>Sensor validity checking</b> | Invalid or out-of-range temperature readings force FAILSAFE                     |
| <b>Irrigation timeout</b>       | Pump cannot run indefinitely during IRRIGATING.                                 |
| <b>Hysteresis</b>               | Prevents flickering near thresholds, such as soil dry/clear or water low/clear. |
| <b>Manual reset/acknowledge</b> | The system only exits failsafe after safe recovery and button input.            |
| <b>Non-blocking timing</b>      | millis() allows continuous sensing, output updates, and timeout supervision.    |

The safety design prioritises protecting the pump and preventing unsafe irrigation. The system always checks water availability and sensor validity before allowing irrigation, and the pump is forced off in all warning and failsafe conditions.

## 7. Code Structure

| <b><i>File</i></b>                                                | <b><i>Purpose</i></b>                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>sketch.ino</b>                                                 | Contains setup() and loop(), giving the main execution flow                                                                                                                                                                                                                                                                        |
| <b>Config.h</b>                                                   | Stores libraries, pin definitions, thresholds, FSM enum, timing variables, sensor variables, and global flags                                                                                                                                                                                                                      |
| <b>Sensors.ino</b>                                                | Reads sensors, maps raw data to percentages, updates hysteresis flags, and controls adaptive polling intervals                                                                                                                                                                                                                     |
| <b>FSM.ino</b>                                                    | Contains state transition logic, cooldown logic, and state name helper                                                                                                                                                                                                                                                             |
| <b>Outputs.ino</b>                                                | Maps FSM states to relay, buzzer, and RGB LED outputs                                                                                                                                                                                                                                                                              |
| <b>Display.ino</b>                                                | Updates the LCD and Serial Monitor                                                                                                                                                                                                                                                                                                 |
| <b>Button.ino</b>                                                 | Handles debounced pushbutton input                                                                                                                                                                                                                                                                                                 |
| <b>DHT.h, Wire.h, LiquidCrystal_I2C.h, Arduino core functions</b> | Reads temperature data from the DHT22 sensor for heat-risk detection. Enables I2C communication between the Arduino Uno and LCD display. Controls the 20x4 I2C LCD used to display live readings, state, and warnings. Uses millis(), analogRead(), digitalWrite(), tone(), and pinMode() for timing, sensing, and output control. |

Main loop flow: button input → adaptive sensor read → hysteresis update → FSM update → output control → LCD/Serial update. As seen in the architecture

Key libraries used include the following: DHT.h for the DHT22 temperature sensor, Wire.h for I2C communication, and LiquidCrystal\_I2C.h for the LCD. Arduino core functions such as millis(), analogRead(), digitalWrite(), tone(), and pinMode() support timing, sensing, and output control. For the basic functionality.

## 8. Key Code Snippets

| <b><i>Key code snippet</i></b>                                                                                                                                                                                                             | <b><i>Purpose / explanation</i></b>                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>enum SystemState { IDLE, MONITORING, IRRIGATING, WARNING, FAILSAFE };</code>                                                                                                                                                         | This enum defines the five possible operating states of the controller.                                                                       |
| <code>unsigned long getSensorInterval() { switch (currentState) { case IDLE: return 2000; case MONITORING: return 1000; case IRRIGATING: return 500; case WARNING: return 300; case FAILSAFE: return 250; default: return 1000; } }</code> | Adaptive behaviour component. The controller checks sensors more frequently in higher-risk states and less frequently in safe or idle states. |
| <code>bool irrigationDemandActive() {return soilDry &amp;&amp; tempHigh &amp;&amp; !waterLow &amp;&amp; !waterFailsafe &amp;&amp; sensorsValid;;}</code>                                                                                   | This demonstrates sensor fusion because irrigation demand depends on soil dryness, temperature, water safety, and sensor validity.            |
| <code>if (!sensorsValid    waterFailsafe) {if (currentState != FAILSAFE) {changeState(FAILSAFE); } return;}</code>                                                                                                                         | This protects the system by immediately entering failsafe if a sensor is invalid or water level is critically low.                            |
| <code>if (!waterLow &amp;&amp; !waterFailsafe &amp;&amp; sensorsValid) {digitalWrite(RELAY_PIN, RELAY_ON);} else {digitalWrite(RELAY_PIN, RELAY_OFF);}</code>                                                                              | Even in the irrigation state, the pump is only allowed to run if water and sensor conditions are safe.                                        |

## 9. Results / Demonstration Summary

The final Wokwi simulation successfully demonstrated the A.I.U operating across all five FSM states. The LCD, RGB LED, buzzer, relay, and the Serial Monitor provided clear feedback for each operating mode and situation.

| <b><i>Test</i></b>                              | <b><i>Expected result</i></b>                  | <b><i>Outcome</i></b> |
|-------------------------------------------------|------------------------------------------------|-----------------------|
| <b>Startup</b>                                  | System begins in IDLE                          | ☑                     |
| <b>Button press</b>                             | Enters MONITORING                              | ☑                     |
| <b>Dry soil + high temperature + safe water</b> | Enters IRRIGATING, pump on                     | ☑                     |
| <b>Irrigation timeout</b>                       | Enters WARNING, pump off                       | ☑                     |
| <b>Low water below 25%</b>                      | Enters WARNING, pump locked off                | ☑                     |
| <b>Critical water below 10%</b>                 | Enters FAILSAFE                                | ☑                     |
| <b>Water recovery above 15% + button</b>        | Returns to IDLE                                | ☑                     |
| <b>Soil hysteresis</b>                          | Dry condition does not flicker near threshold  | ☑                     |
| <b>Adaptive polling</b>                         | Serial Monitor shows changing sensor intervals | ☑                     |
| <b>Night heat-risk</b>                          | Lower heat threshold used at night             | ☑                     |

Some limitations for the project and challenges encountered were the Wokwi uses simulated sensor values rather than real agricultural hardware. As the sensors don't replicate erratic behaviour. The soil moisture and water level are represented by potentiometers. The pump is represented by a relay module rather than a physical irrigation pump, which does not reflect aspect such as water pressure/flow. Also, the threshold values are chosen for ideal demonstration and would need calibration for real-world deployment. On top of this, the long and difficult compilations slowed down development.

## 10. Individual Contributions

| <b><i>Member</i></b> | <b><i>Contribution</i></b>                                                                                                                                                       |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thomas Dosbon        | Designed the A.I.U concept, built the Wokwi circuit, implemented the FSM, sensor processing, adaptive polling, safety logic, output control, testing, report, and demonstration. |

